“UNIVERSIDAD PRIVADA FRANZ TAMAYO”
FACULTAD DE INGENIERIA



PRACTICA GIT

DOCENTE:

ING. FERNANDEZ QUISBERT PATRICIA

INTEGRANTES:

SOLDADO QUISPE CARLOS DANIEL

TUMIRI BARRIONUEVO JUAN CARLOS

MATERIA:

PROYECTO INTEGRADOR INTERMEDIO II

CARRERA: INGENIERIA DE SISTEMAS

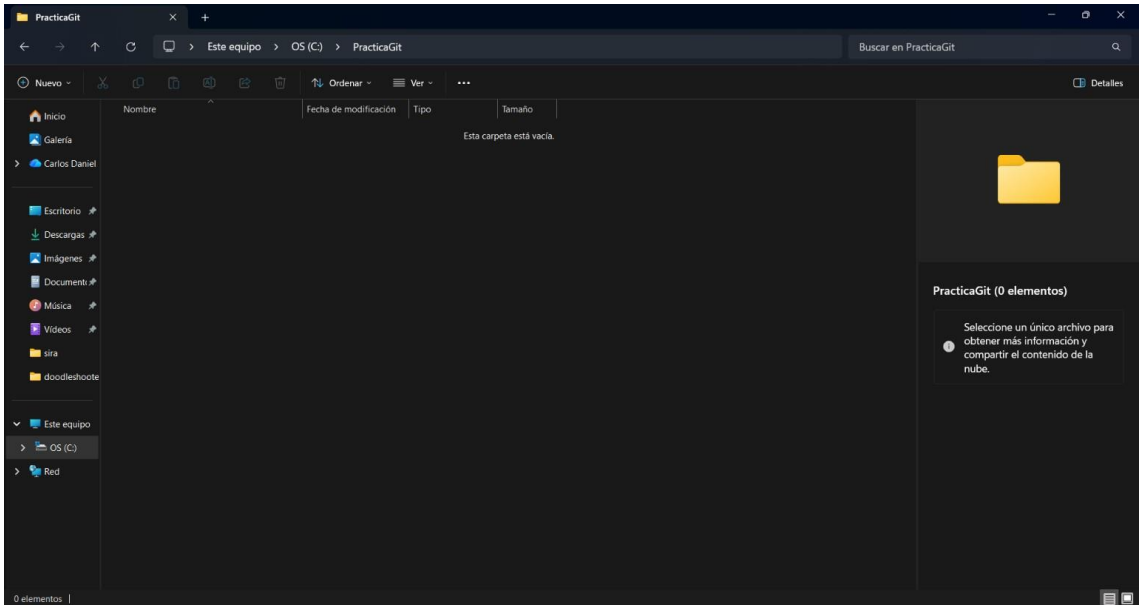
FECHA DE ENTREGA: 08/10/2026

A. CREAR UN REPOSITORIO

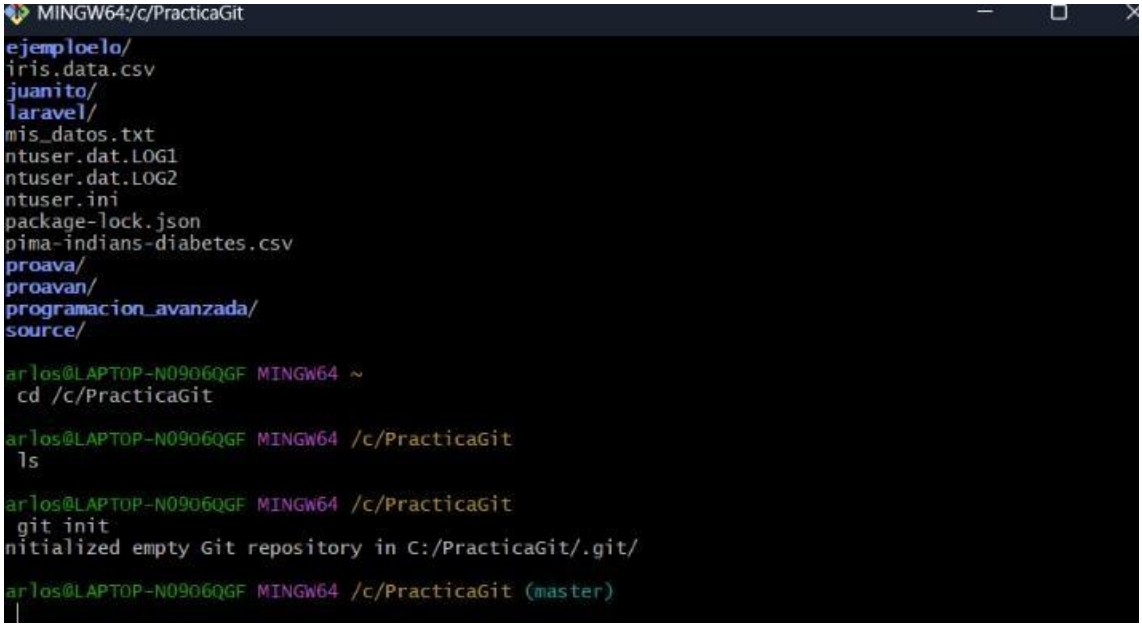
1. Cree una carpeta llamada: PracticaGit



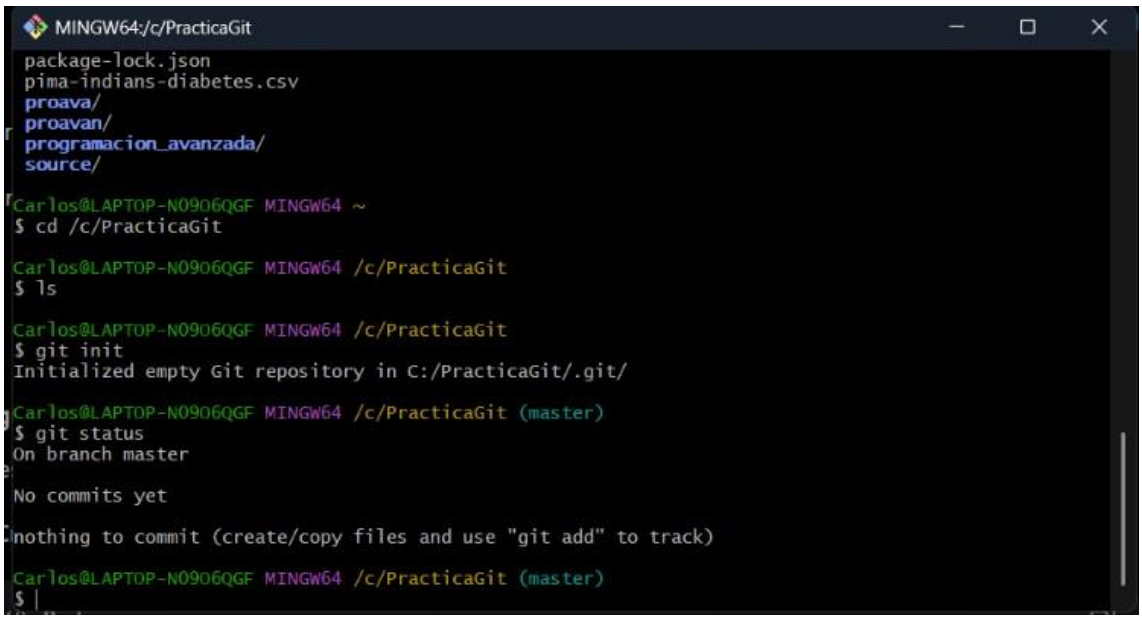
2. Ingrese a la carpeta.



3. Inicialice un repositorio Git.

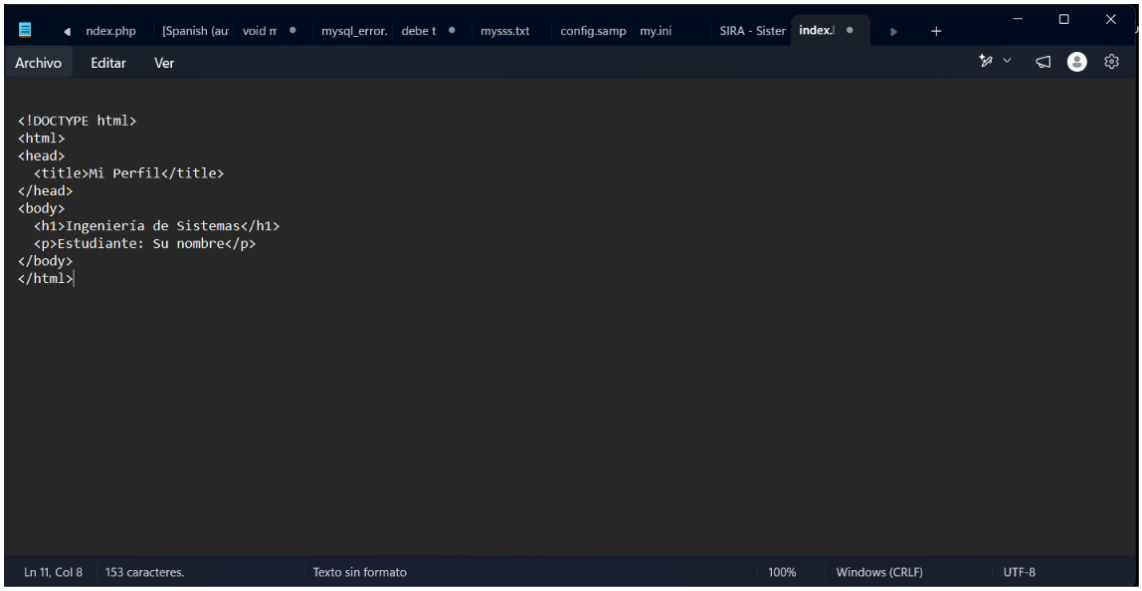


4. Verifique el estado del repositorio.

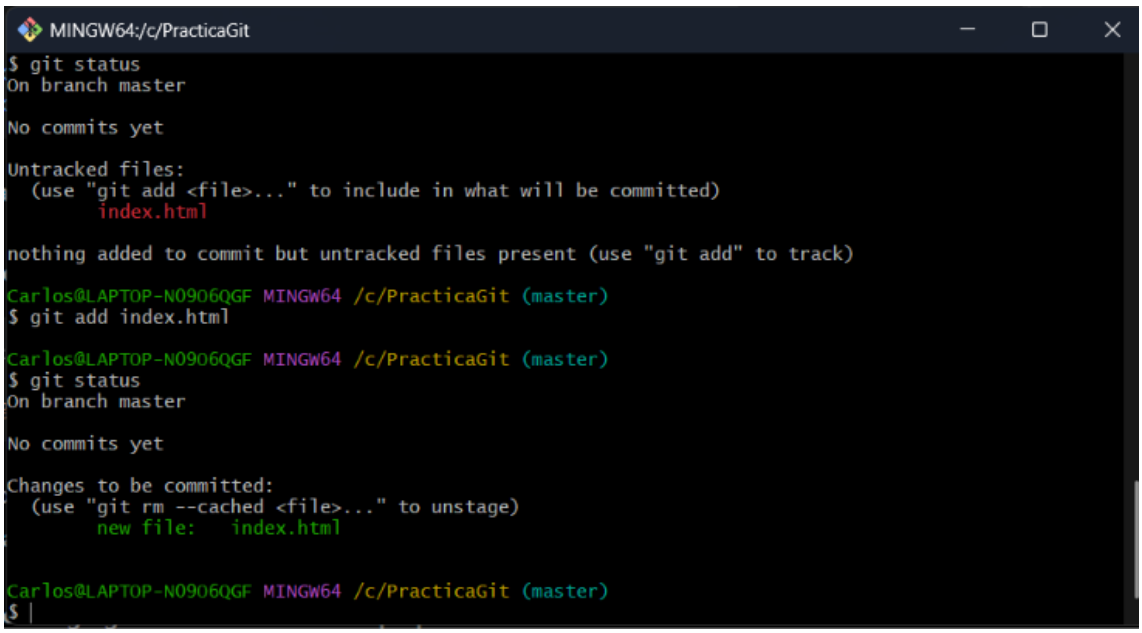


B. Área de trabajo y primer commit

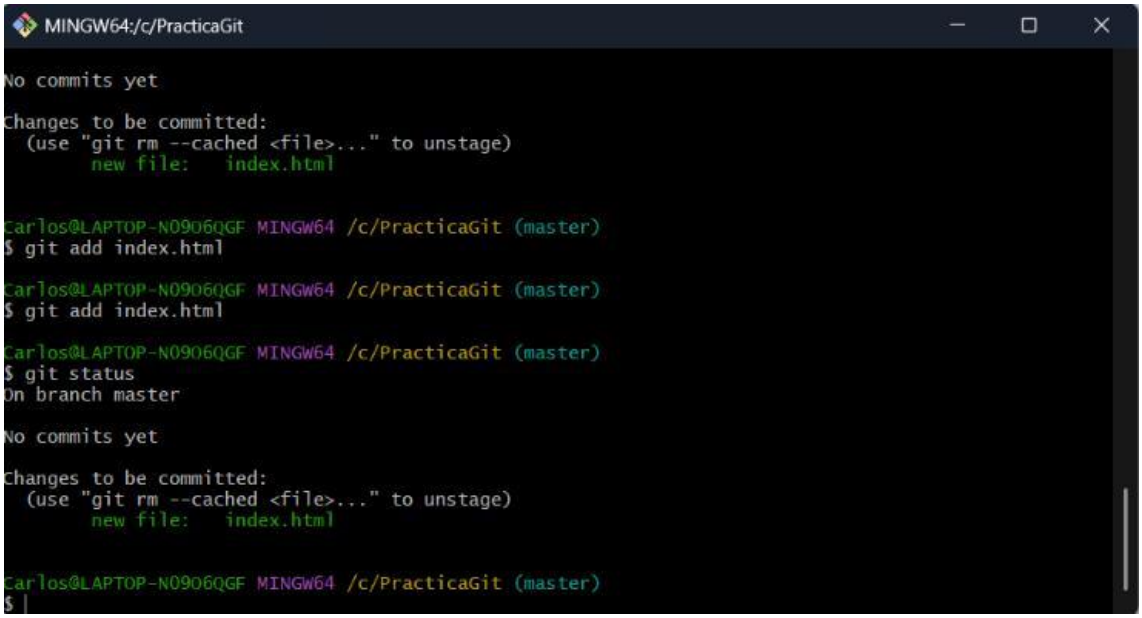
5. Cree un archivo llamado: index.html con el siguiente contenido:



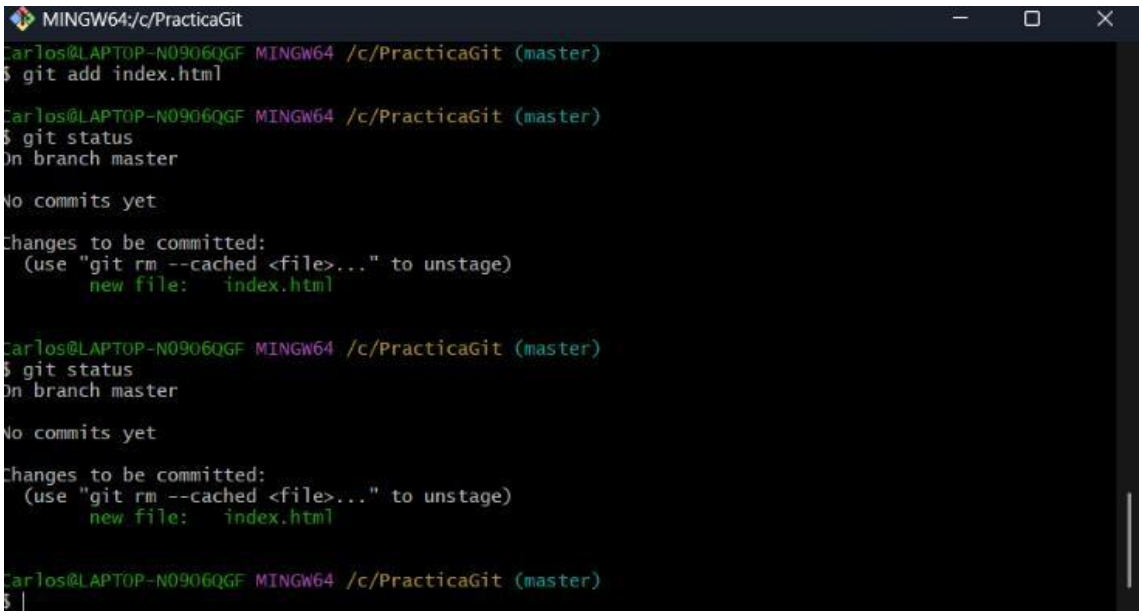
6. Consulte el estado del repositorio.



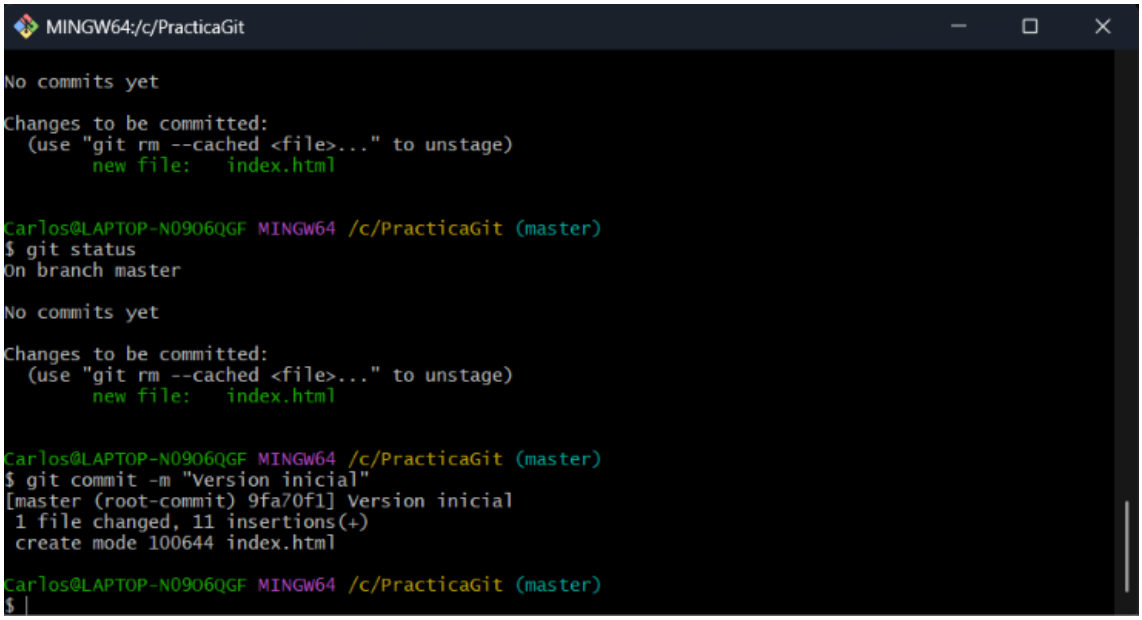
7. Agregue index.html al área de preparación (staging area).



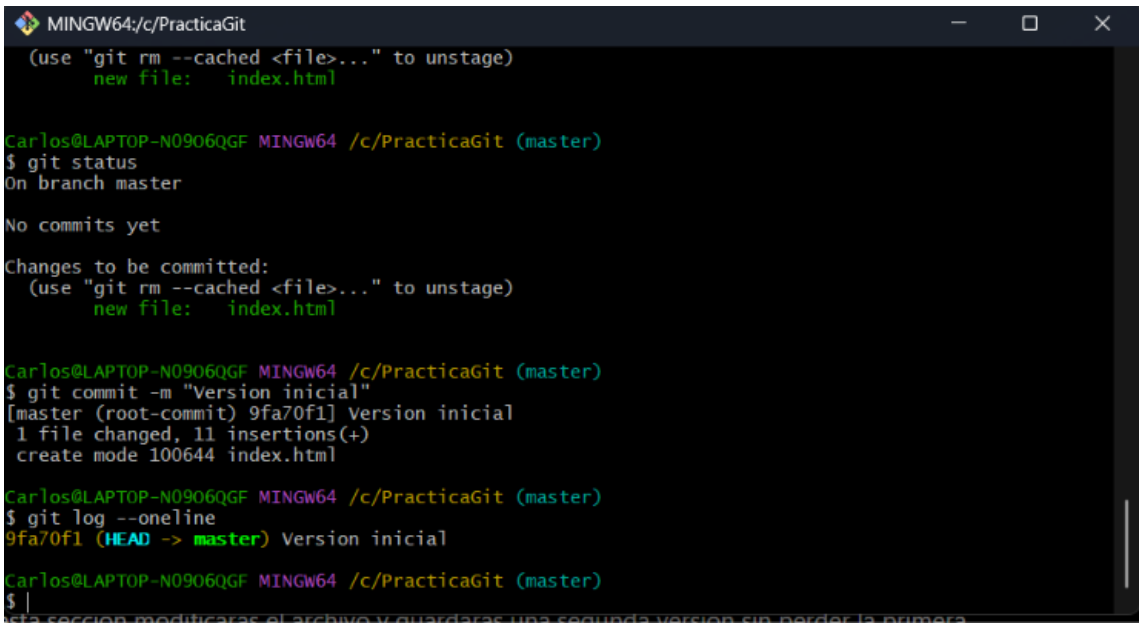
8. Consulte nuevamente el estado.



9. Realice el primer commit con el mensaje: “Version inicial”

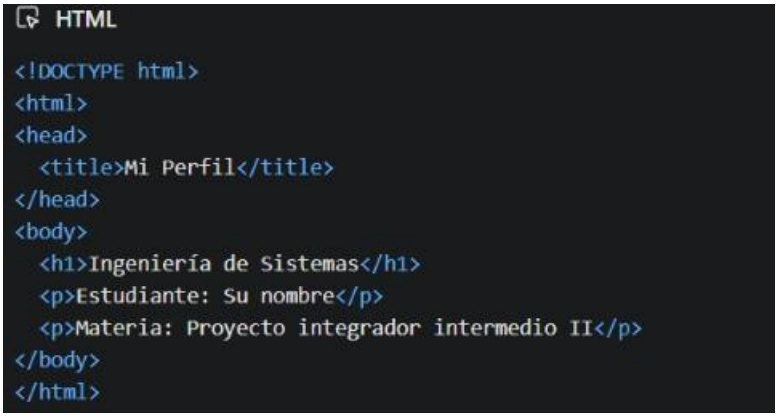


10. Consulte el historial.

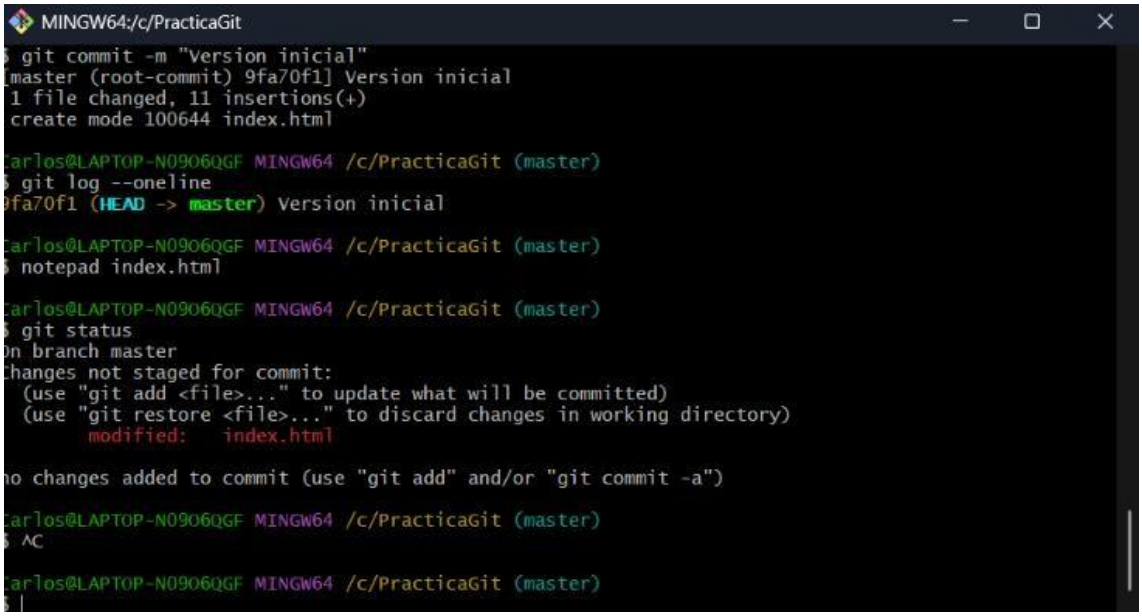


C. Modificar y guardar una nueva versión

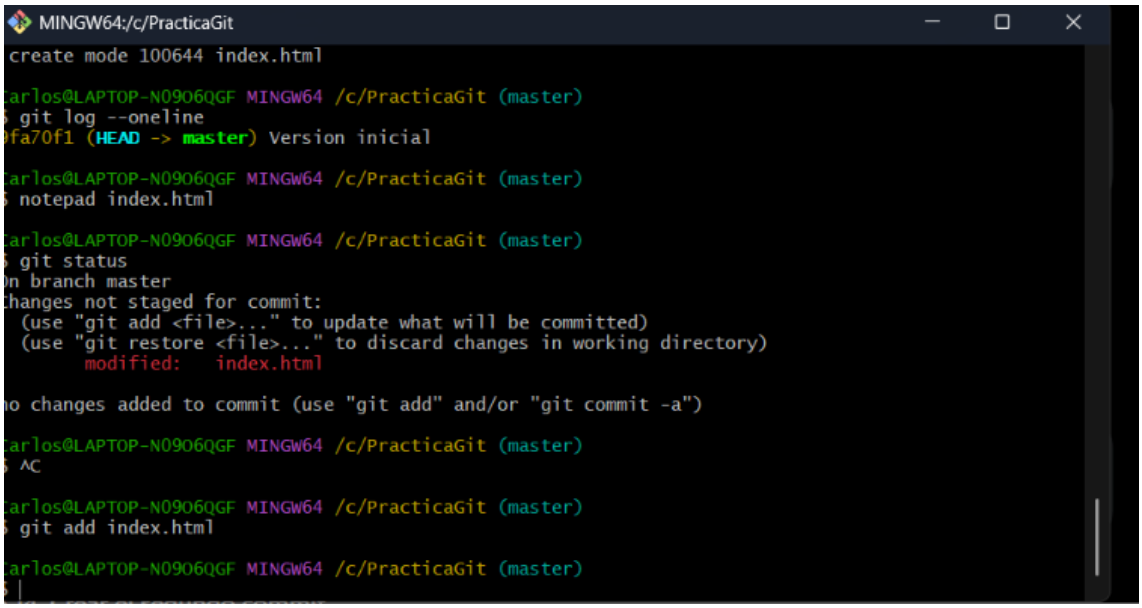
11. Modifique index.html y agregue debajo del párrafo: Materia: Proyecto integrador intermedio II



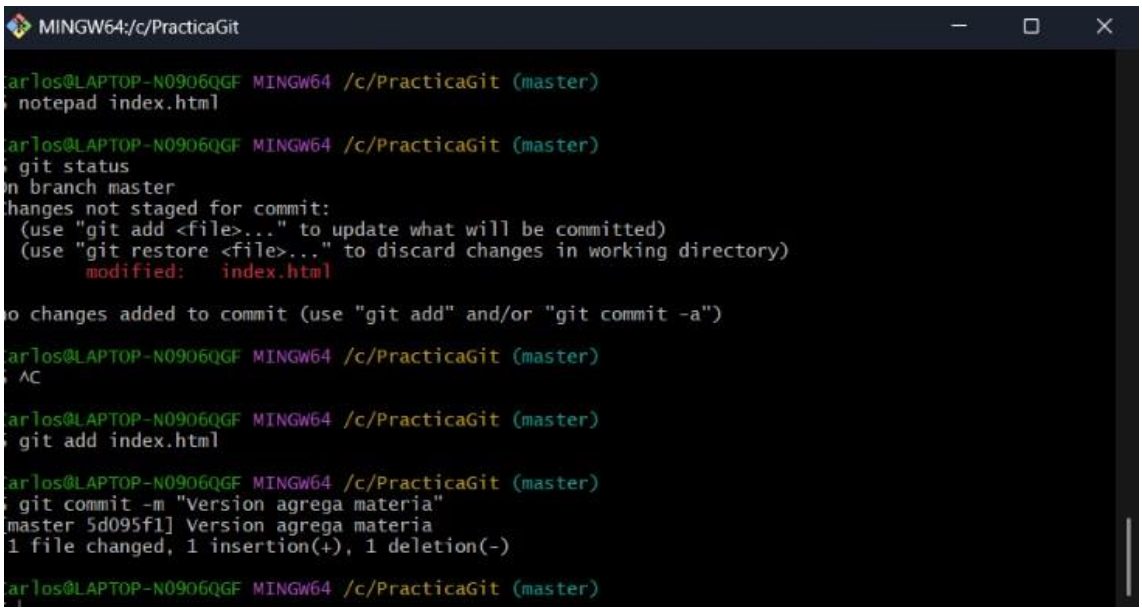
12. Consulte el estado.



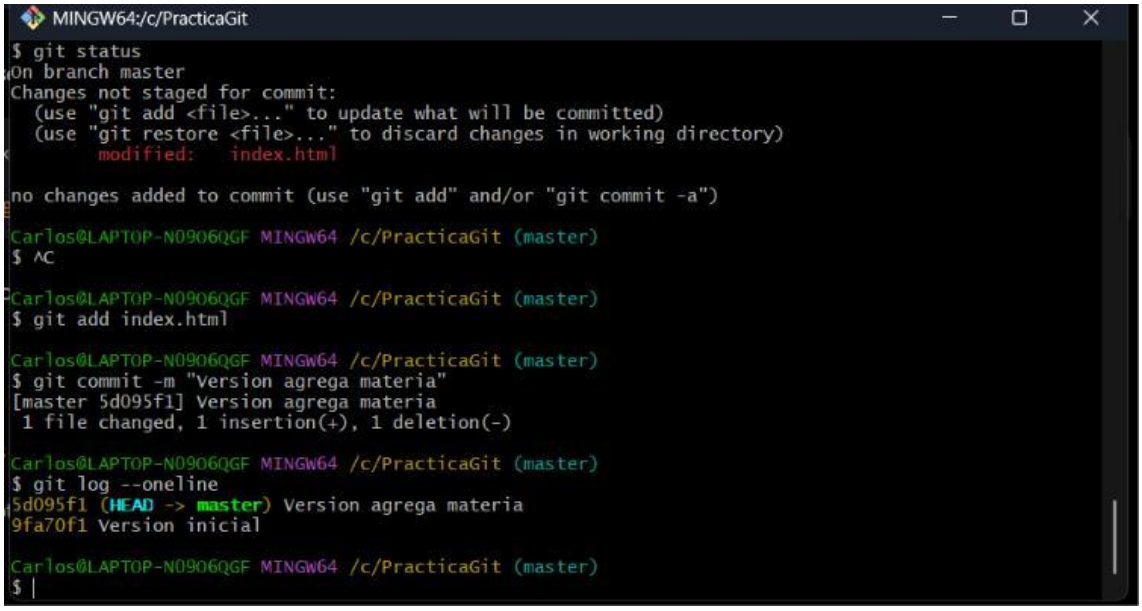
13. Agregue el cambio al staging area.



14. Realice un nuevo commit con el mensaje: “Version agrega materia”

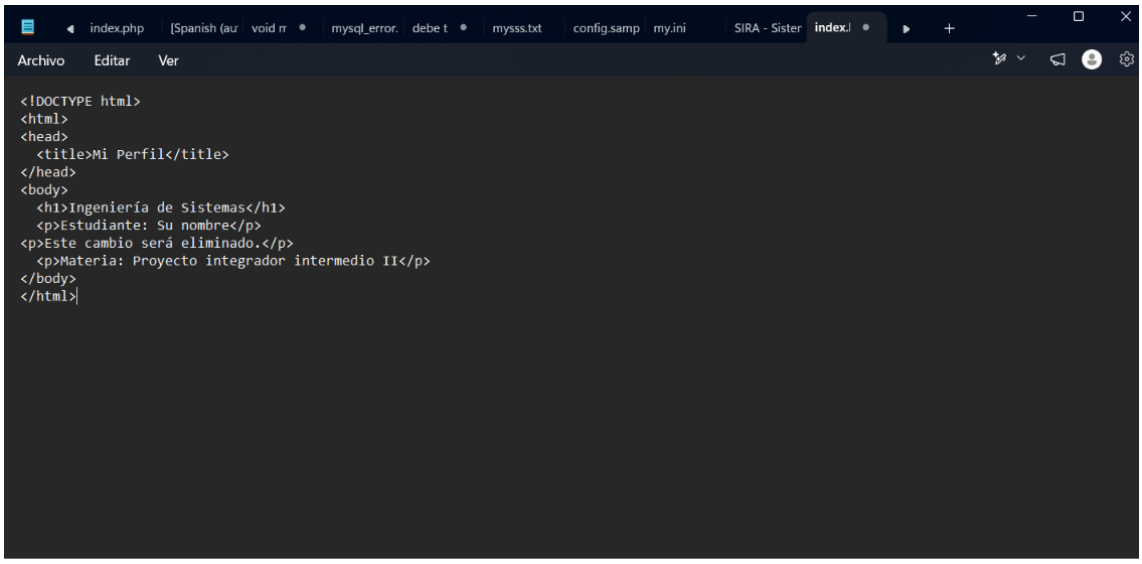


15. Consulte el historial.

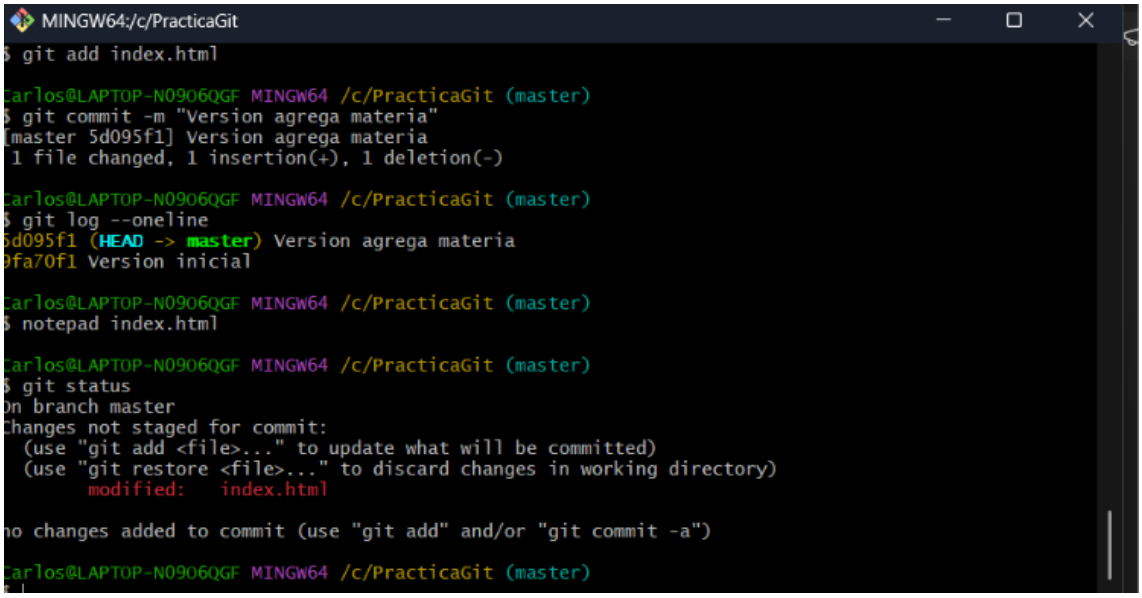


D. Practicar restore

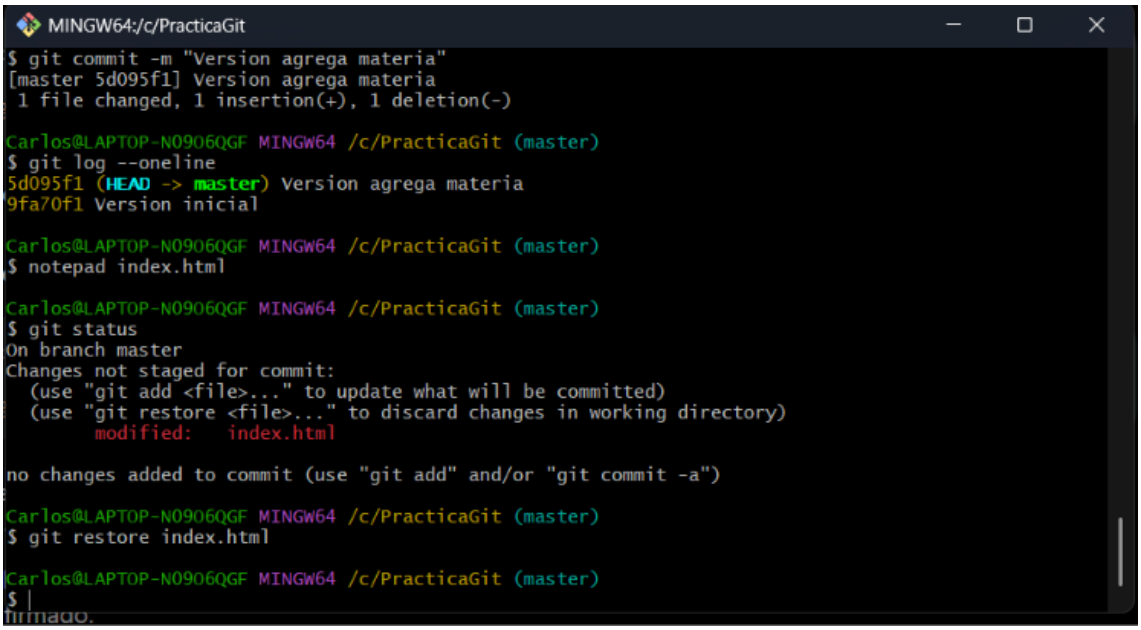
16. Modifique nuevamente index.html agregando: Este cambio será eliminado.



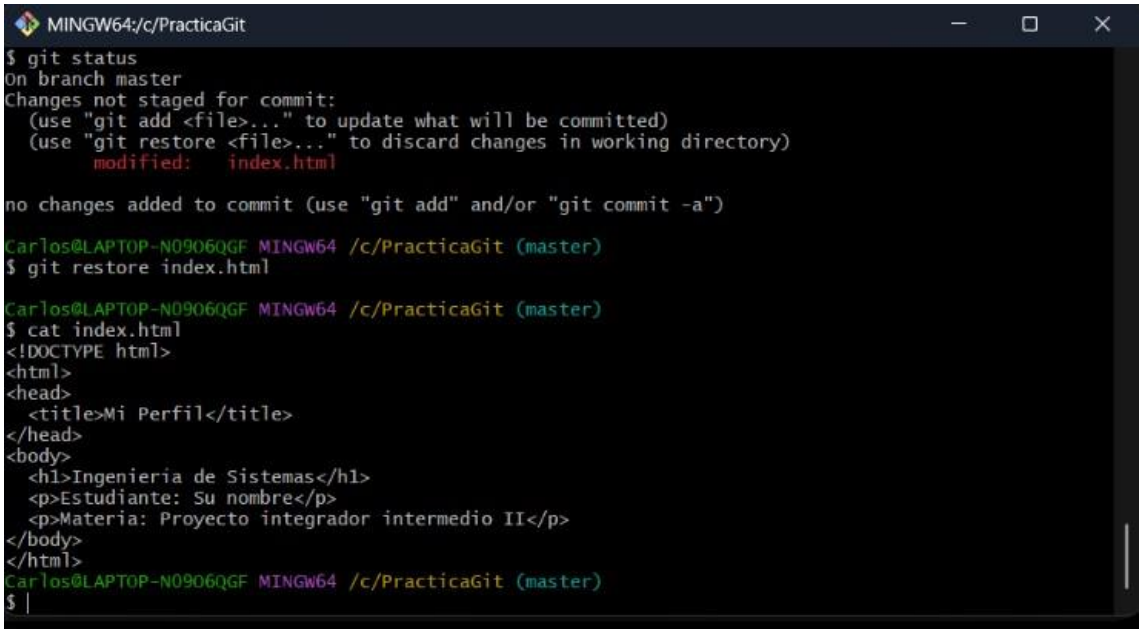
17. Consulte el estado.



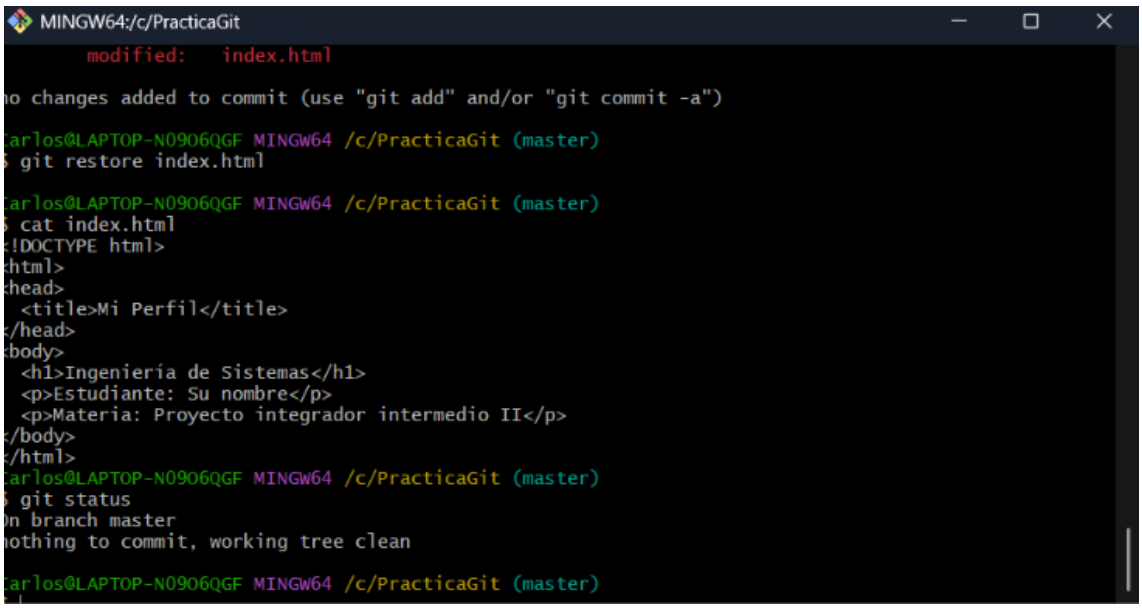
18. Use: git restore index.html



19. Compruebe que el cambio desapareció.



20. Consulte el estado del repositorio.

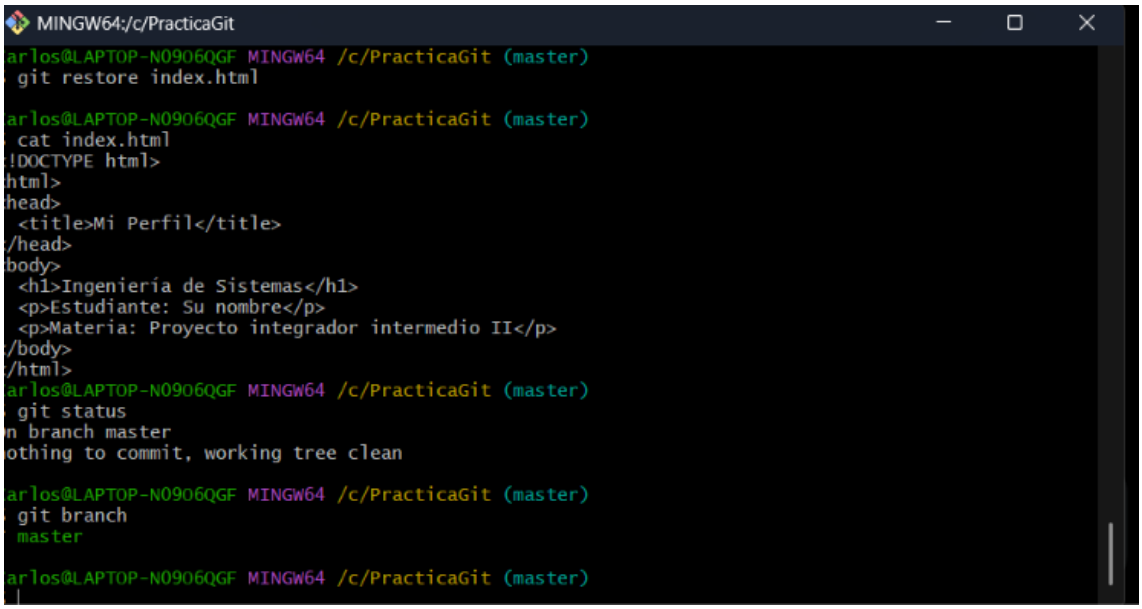


Pregunta: ¿Cómo recupera este archivo? Origen, Destino Seleccione entre: área de trabajo, área de preparación, repositorio

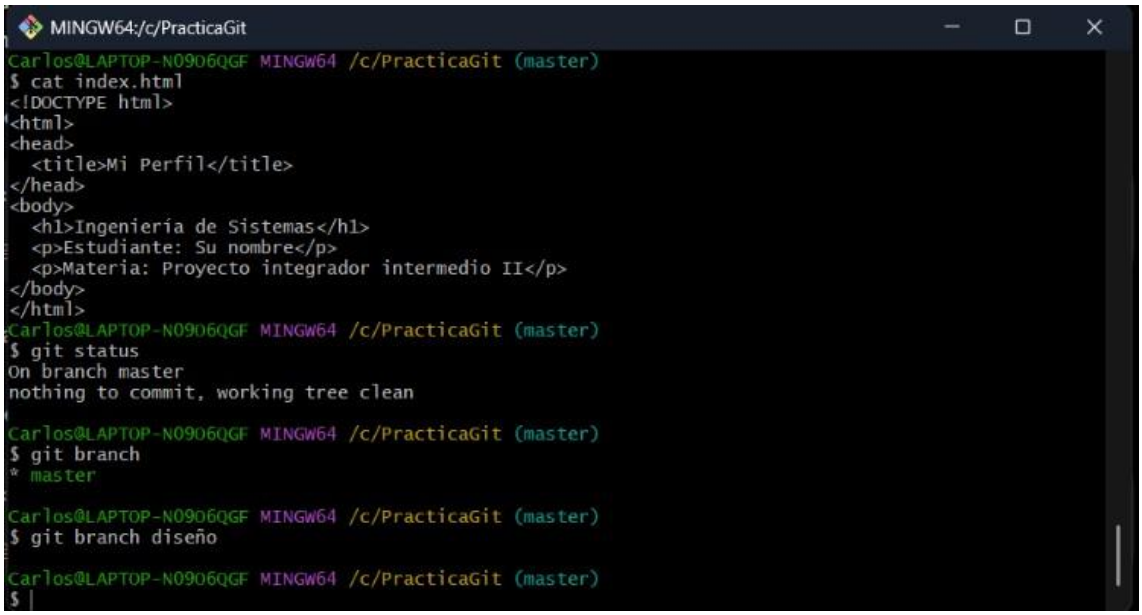
Origen: Repositorio
Destino: Área de trabajo

E. Crear y utilizar una rama

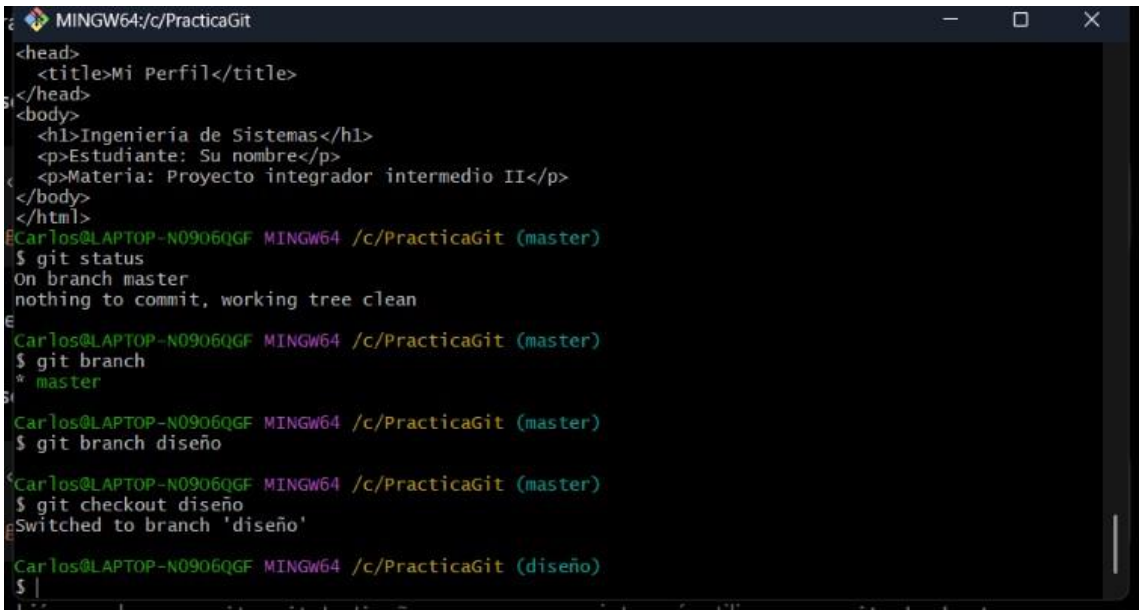
21. Consulte las ramas existentes: git Branch



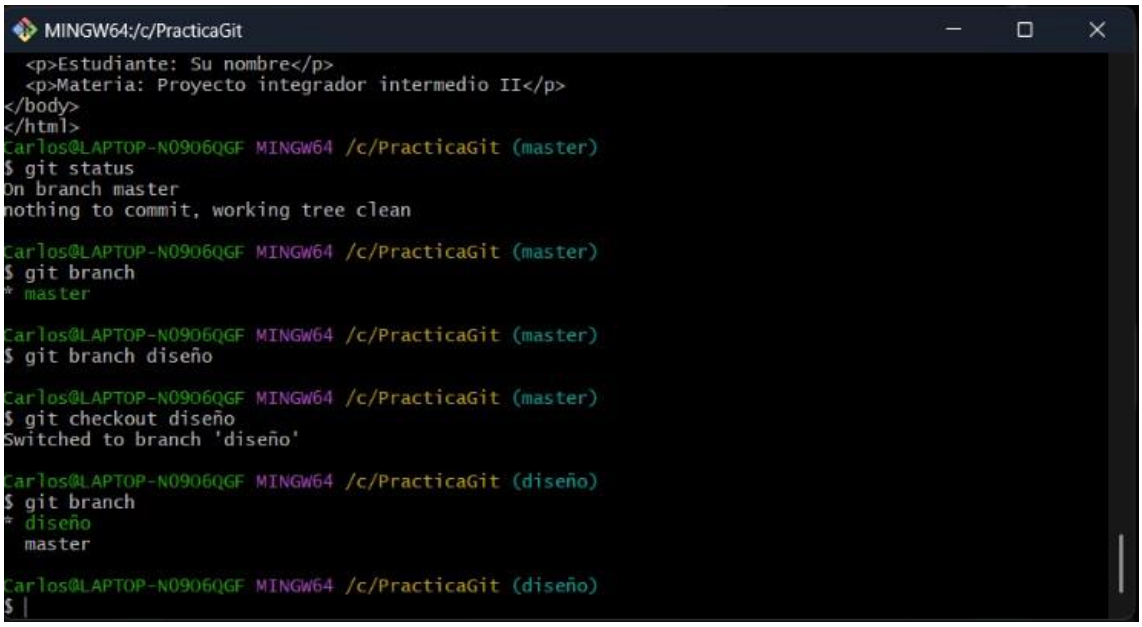
22. Cree una rama llamada: diseño



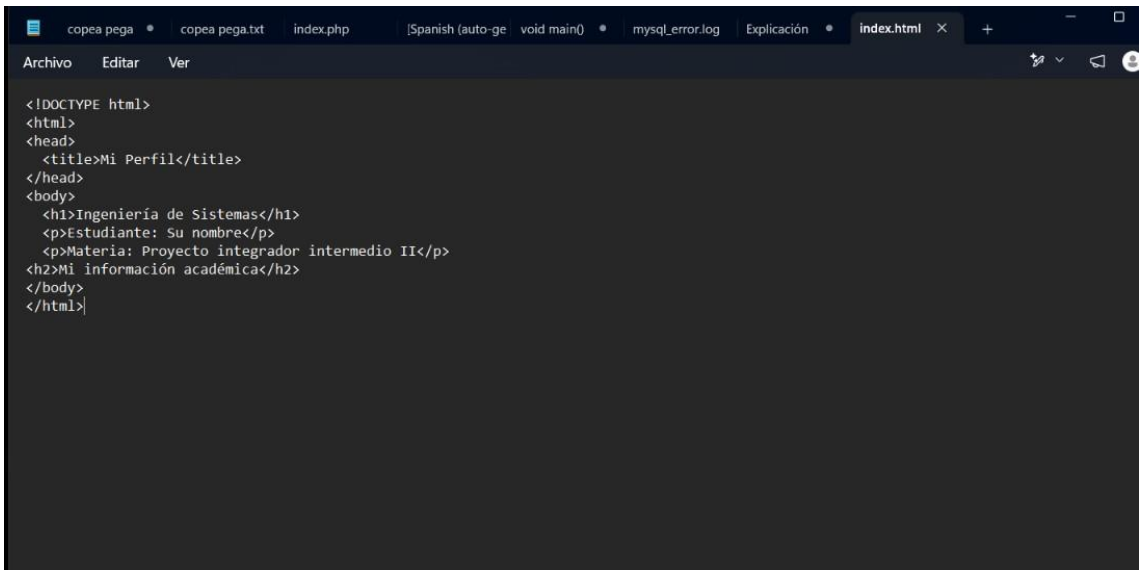
23. Cambie a esa rama. Puede hacerlo con: git checkout diseño



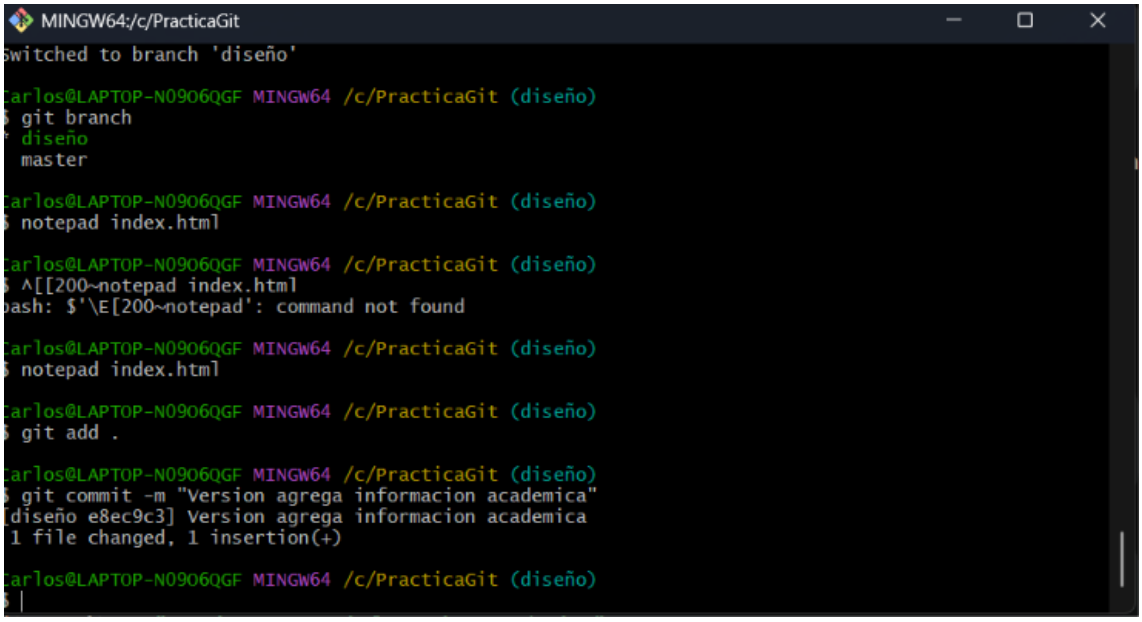
24. Compruebe en qué rama se encuentra.



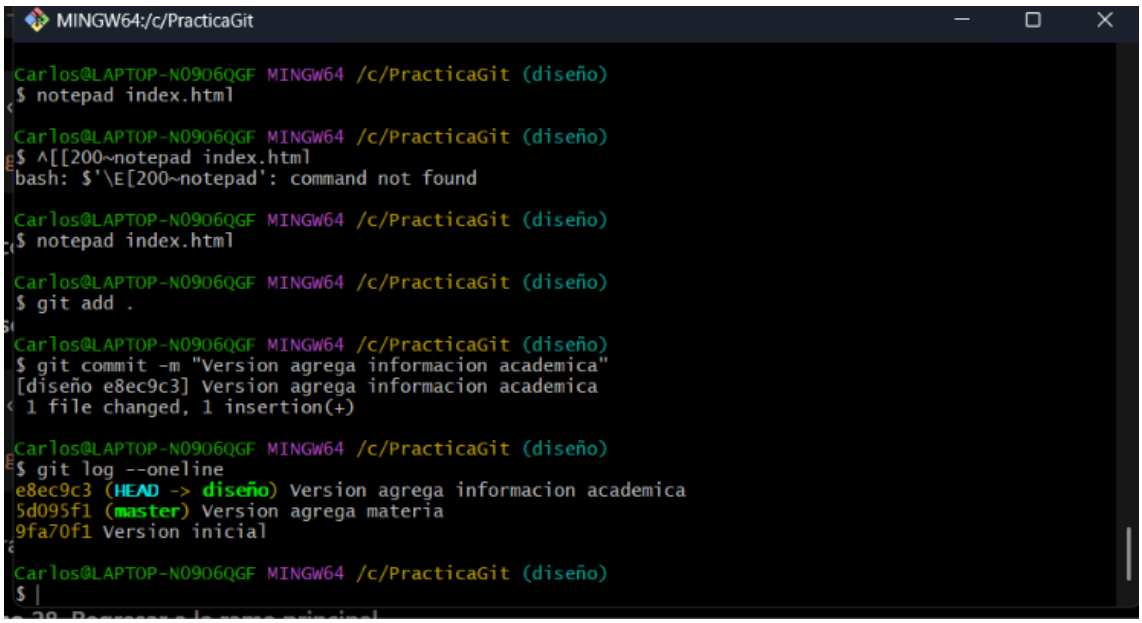
25. Modifique index.html agregando: <h2>Mi información académica</h2>



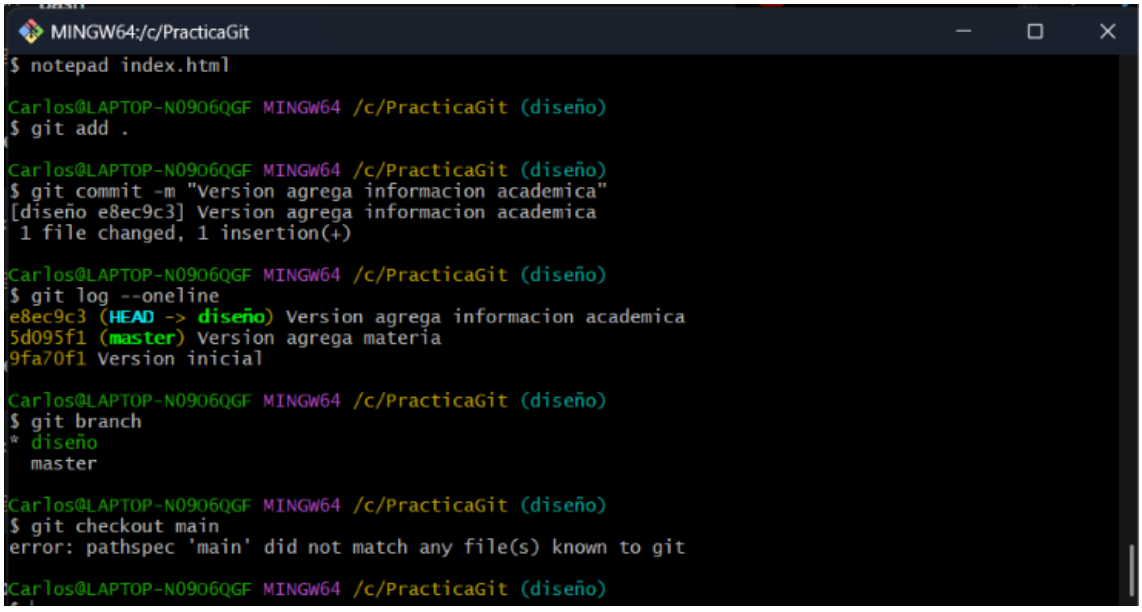
26. Realice: git add . git commit -m "Version agrega informacion academica"



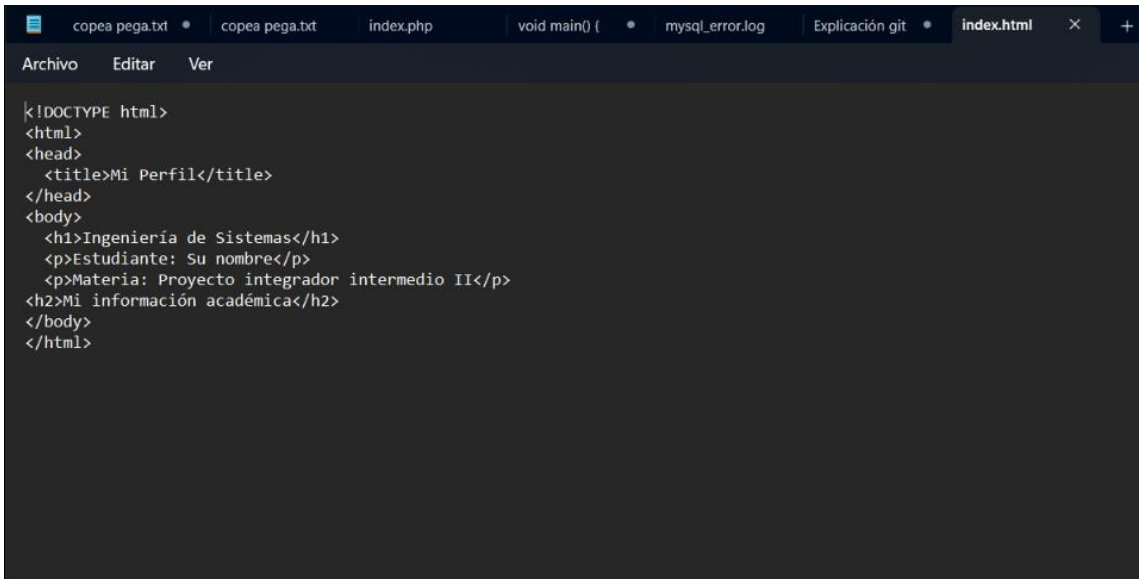
27. Consulte el historial.



28. . Regrese a la rama principal: git checkout master Si Git creó la rama principal con el nombre main, utilice: git checkout main



29. Abra index.html. Observe: el cambio realizado en la rama diseño no aparece en la rama principal

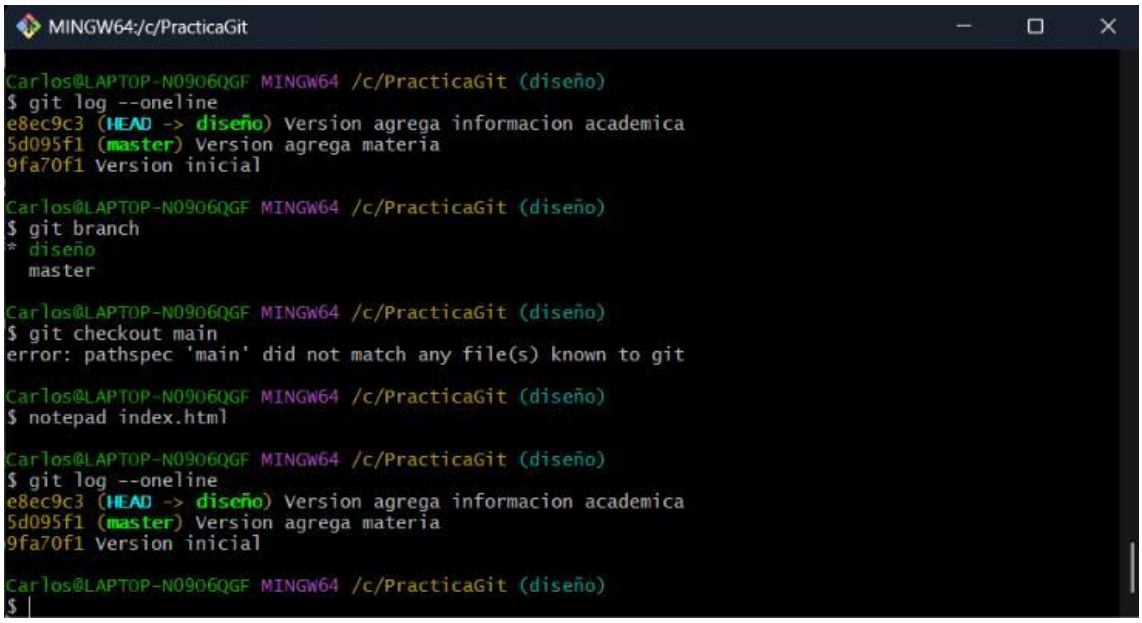


Pregunta: ¿Por qué la modificación realizada en diseño no aparece en master/main?

R. Porque diseño y master/main son ramas independientes. El commit se guardó en diseño, por lo que la rama principal no incorpora automáticamente ese cambio. Para que aparezca en la rama principal, habría que integrar los cambios mediante una operación como git merge.

F. Recuperar información del historial

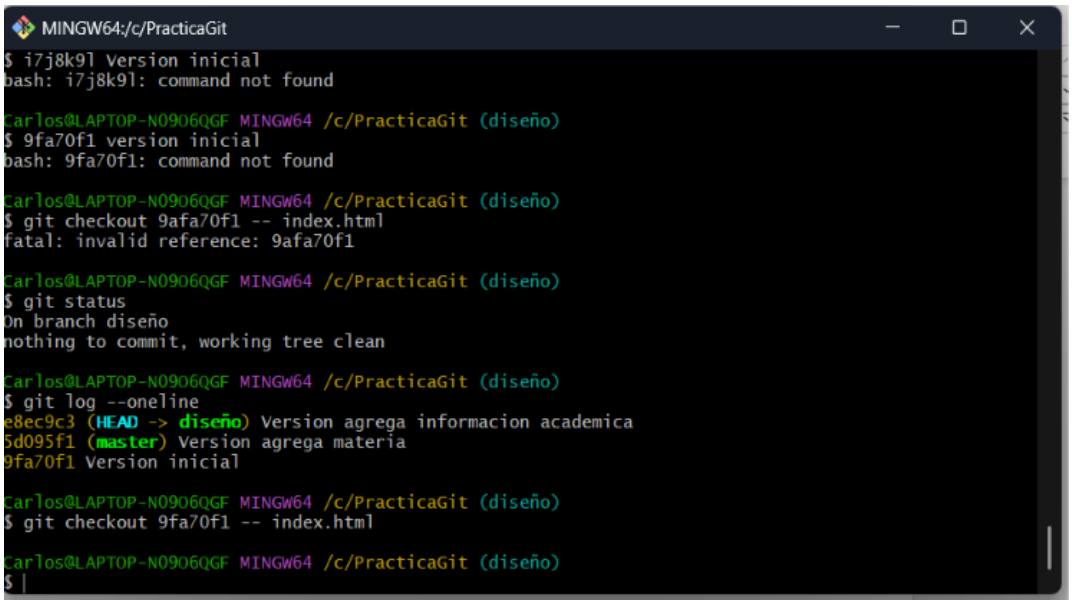
30. Ejecute: git log –oneline



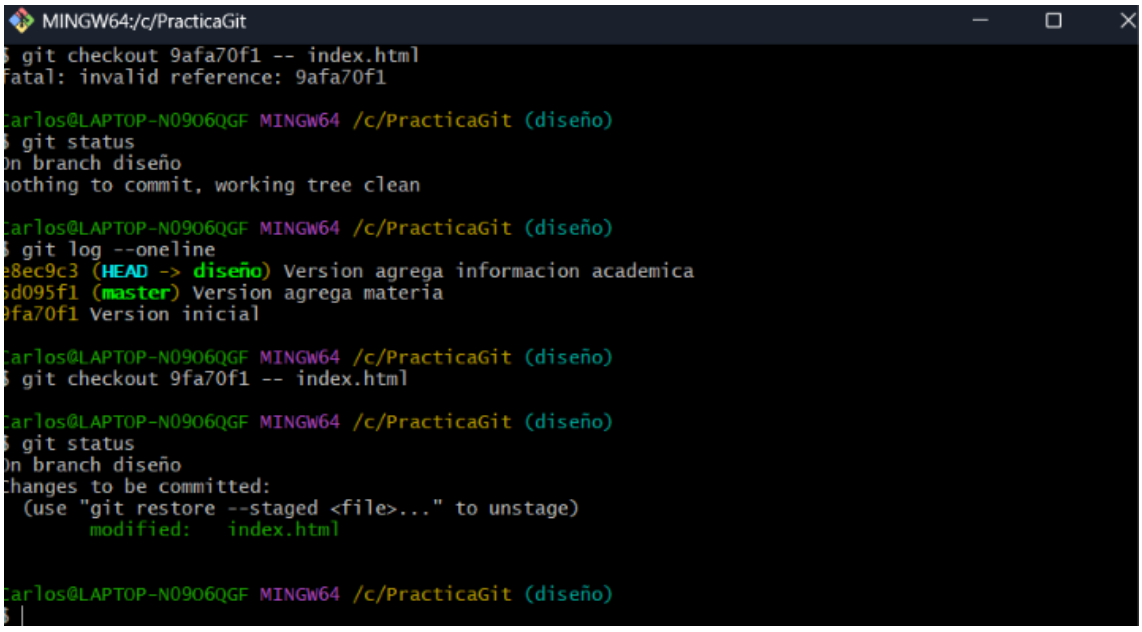
31. Identifique el código del commit correspondiente a:Version inicial



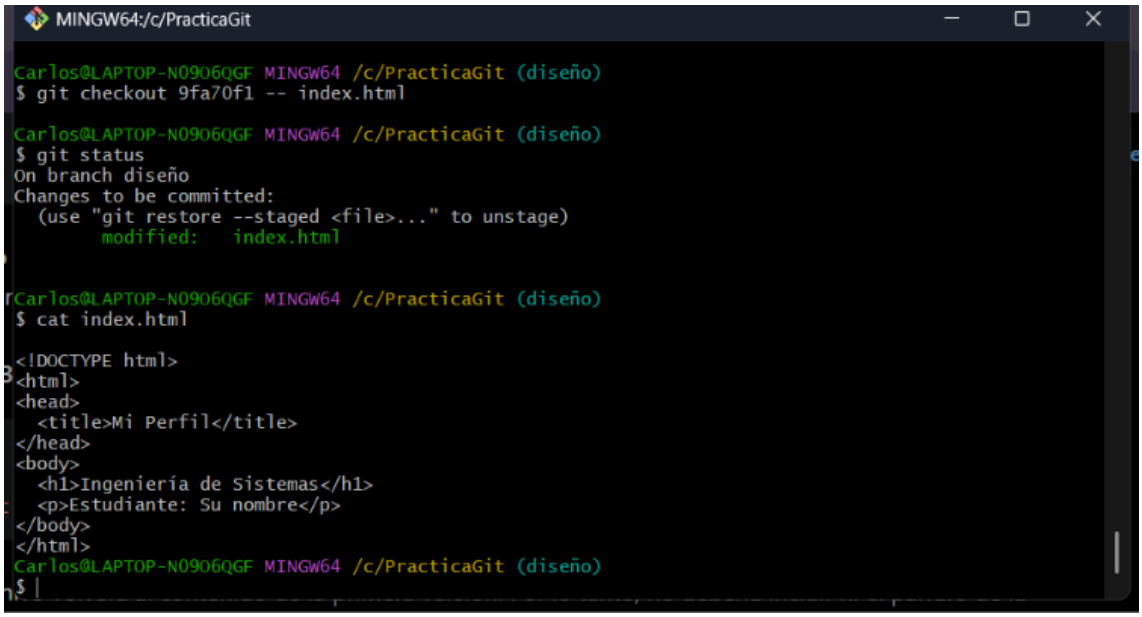
32. Use: git checkout CODIGO_COMMIT -- index.html Reemplace CODIGO_COMMIT por el código correspondiente.



33. Consulte el estado: git status



34. Observe qué ocurrió con index.html.



Pregunta: ¿Qué ocurrió con index.html? Explique

R. El archivo index.html fue recuperado de la versión identificada por el commit

Version inicial. Por eso desaparecen del archivo las modificaciones posteriores. Git prepara esa restauración en el área de preparación, pero no elimina los commits posteriores ni borra el historial del repositorio.

Preguntas finales. Responder de forma concreta:

1. ¿Cuál es la diferencia entre área de trabajo y staging area?

R. El área de trabajo es donde se crean y modifican los archivos del proyecto. El staging area es el área donde se preparan los cambios que se incluirán en el próximo commit.

2. ¿Para qué sirve git log --oneline?

R. Sirve para consultar el historial de commits de forma resumida, mostrando el código corto de cada commit y su mensaje en una sola línea.

3. ¿Qué diferencia existe entre git restore y git checkout?

R. git restore permite descartar modificaciones o recuperar archivos desde otra área o versión. git checkout permite cambiar de rama o recuperar archivos de un commit específico, como se hace en esta práctica.

4. ¿Para qué sirve una rama (branch)?

R. Una rama sirve para desarrollar cambios de forma independiente sin afectar directamente la rama principal. Permite trabajar en nuevas funciones, probar modificaciones y después integrar los cambios cuando corresponda.